

Configuration Generation Is Not Authorization: Review Invariants and Enforcer Independence

Configuration Generation Is Not Authorization: Review Invariants and Enforcer Independence I

An AI agent can write a declarative configuration, an access-control policy, or a deployment change as fluently as it writes code. Generation is not the security event. The security event is whatever independently enforces the boundary between what the agent may change and what it may only request — above all, the mechanisms that constrain the agent itself. This section deepens that argument, states the 9 review invariants that an agent must never be able to violate, and locates the separation of policy writing, policy approval, and permission exercise inside the authority architecture of sec. 10.

The distinction is complete mediation (Saltzer and Schroeder 1975): every access to a protected resource is checked by an enforcement point, and the authorship of the request is irrelevant to whether the check runs. A proposed configuration is a request. Declarative formats make requests legible — diffable, reviewable, testable — which is why they are attractive in this threat model; but legibility is a property of the document, while enforcement is a property of the system that acts on it. A policy nobody enforces is a wish. An enforced policy whose enforcement mechanisms the agent can reconfigure is the same wish, one step removed, and it is the more dangerous form because it advertises control that does not exist.

Generation versus enforcement I

The authorized-misuse path of sec. 3 runs straight through configuration. An agent need not disable isolation through an exploit if it can propose a configuration generation that disables isolation and have that generation applied as routine system management. The attack is a policy review that did not happen, not a boundary that was crossed.

Declarative configuration as an inspectable policy surface I

Used correctly, declarative configuration is the strongest inspectable policy surface in common use. The NixOS model makes intended system state explicit, keeps generations diffable, and supports activation and rollback of configuration as a first-class operation (community 2026a, 2026b). Review can compare what will be deployed against what was approved; drift becomes visible; policy becomes data. These are real gains over imperative mutation, and this review's recommendations in sec. 6 lean on them.

Declarative configuration as an inspectable policy surface I

Two traps sit alongside the gains. The first is inspection without enforcement: reviewing the file changes nothing if the deployment pipeline applies whatever appears next. The second is enforcement without independence: a reproducibly deployed privilege escalation remains a privilege escalation, and reproducibility makes it deterministic and fleet-wide — every machine receives the escalation in the same rebuild. Reproducible deployment guarantees the same result everywhere; it does not ask whether the result is authorized (Project 2026a). Credential handling shows the same trap from the secrets side: declarative cleanliness that embeds secrets in configuration exposes them to every store reader and possibly to external caches, so the documented practice reads secrets at runtime under access control (Project 2026b). Inspection value and authorization value are different properties, and the second never follows from the first.

Declarative configuration as an inspectable policy surface I

Provenance inspection, in particular, is maturing from a manual discipline into packaged tooling, and the Nix ecosystem is where it is furthest along. Source provenance is now a required attribute in `nixpkgs`: a `meta.sourceProvenance` declaration tagging each package's inputs — from official releases to unvendored source trees — is merged as a requirement rather than a convention, which makes the provenance question answerable from the derivation data itself rather than from documentation (community 2026c). Around that requirement a tooling stack has formed: `sbomnix` generates CycloneDX and SPDX software bills of materials for a NixOS closure and emits SLSA provenance attestations for builds ((TII) 2026); Trustix rebuilds packages independently and compares the results bit-for-bit against what the binary cache distributed, converting trust in a substitute into a checkable claim (community 2026d); and the

Declarative configuration as an inspectable policy surface I

`nix provenance show` and `nix provenance verify` subcommands packaged by Determinate Systems expose the provenance of a built artifact for inspection and verification as ordinary command-line operations. The direction is exactly right for this threat model: the provenance of deployed software becomes inspectable data, in the same way declarative configuration made intended state inspectable. The section's caution nonetheless applies unchanged. Provenance attests how an artifact was built and from what inputs; it does not authorize deployment, and a perfectly attested artifact is not thereby a permitted one. Inspection tooling raises the quality of the review; the authorization decision remains with the enforcement points this section describes.

The review invariants I

tbl. 1 states the review invariants: what an agent must never be able to do, the failure each invariant closes, and where enforcement must live. Formally, each invariant is an unreachable-state predicate (**(def:invariant_predicate?)**): no state reachable from agent-writable configuration may violate an invariant eq. 1.

Each review invariant ι_i is an unreachable-state predicate — no state reachable from agent-writable configuration may violate it:

$$\forall g \in G_{\text{agent}} : \neg \text{reachable}(g, \text{violate}(\iota_i)) \quad (1)$$

The review invariants I

Table 1: The 9 configuration review invariants: the capability closed, the failure each prevents, and the enforcement point

Invariant: the agent must never be able to	Failure the invariant closes	Enforcement point
Add an unrestricted credential provider	Every task gaining reach into secrets through one configuration edit	Credential mediation service; provider registry outside agent-writable configuration
Disable isolation	A prevention failure becoming whole-machine compromise by policy rather than exploit	Isolation baseline outside agent-writable generations; changes require external approval and a verified boot path
Widen its own network policy	The egress boundary rewritten toward attacker-chosen destinations, reopening data-loss paths (sec. 12)	Egress policy owned by a boundary the agent cannot rewrite
Replace the trusted update signer	Supply-chain takeover achieved as ordinary configuration change	Signing keys and trust anchors held outside every agent-writable generation
Alter the approving identity	An attacker redefining who counts as an approver, then self-approving	Approving-identity registry immutable from any agent context
Mount unrelated user data	Cross-domain reads into <code>personal_id</code> entity or <code>credential_service</code> data	Mount policy scoped per trust domain (tbl. 8)
Expand its own tool grants	Self-issued authority through the tool-bridge surface (sec. 12)	Grant registry held and reviewed as explicit authority grants
Rewrite the audit trail	Destruction of the record incident response depends on	Append-only audit storage outside every execution environment (tbl. 9)
Approve its own policy changes	Closure of the loop from proposal to authorization to exercise	Write-policy / approve-policy / exercise separation (below)

The review invariants I

The 9 invariants share one design principle: the state that would relax an invariant is never reachable from agent-writable configuration. They are capability invariants, not instruction invariants. A well-prompted agent and a compromised agent must be equally unable to violate them, because the authorized-misuse path does not distinguish persuasion from compromise at the enforcement layer. An invariant that holds only while the agent behaves is not an invariant; it is a preference. Biba's integrity model gives the principle its classical form: a subject may not write to an object at a higher integrity level (Biba 1977) — agent-writable configuration is the low-integrity subject, the policy surface that authorizes the system's behavior is the high-integrity object, and generation-is-not-authorization is precisely the prohibition on that write-up.

Separating policy writing, policy approval, and permission exercise I

The write-policy / approve-policy / exercise-permission separation maps onto the authority ladder of sec. 10: proposal and staging are distinct rungs from authorization, and exercise is distinct from both. The operating rule is that no single principal holds more than one of the three roles for a consequential change. The identity that writes a policy change differs from the identity that approves it, and whatever exercises the new permission does so only after an approval that neither writer nor exerciser controls. Hardy's confused deputy is precisely the closure of this loop — the program that compiles the request and holds the authority over the resource it names (Hardy 1988) — and the separation is the structural answer: split the deputy into roles that cannot be merged by the party under

attack.

Deterministic policy checks can automate approval where the check is complete: binary, fully specified, and independent of the task's success. Where judgment is required, the approver receives the full operation context — artifact, destination, scope, expiry — not a summary, mirroring the approval-context discipline of sec. 13. An approver who sees “network policy update” has approved nothing; an approver who sees the exact diff that widens egress to a named destination has at least been given the choice.

Separating policy writing, policy approval, and permission exercise I

Signer management is the sharpest instance of the separation. A configuration change that replaces the trusted update signer — or the signer of a binary cache — converts every subsequent update into attacker-chosen code, so the review must ask who holds signing authority rather than merely whether signatures verify; trusting a signer is not proof that an artifact was produced by the expected derivation (community 2026e). Invariant 4 in tbl. 1 is therefore enforced by construction: signing trust anchors live outside every agent-writable generation, and changing them is the highest-consequence generation request an agent can make — which is exactly why it must be one the agent cannot complete alone.